

Design of Suspension Software Suite, Based on Nonlinear Kinematics
and a Nonlinear 4-Wheel Dynamics Model for Mississippi State's
Bulldog Motorsports

Colin Spilker

Fall 2025

Contents

1 Introduction	3
1.1 Description	3
1.2 Literature Review	3
1.3 Purpose, Benefits, and Objectives	3
2 Geometry Modeling	5
2.1 Kinetic Modeling	5
2.2 Springs and Wheel Rates	11
2.3 Ackermann and Steering Modeling	15
3 Dynamics	20
3.1 Nonlinear Tires	20
3.2 2-Wheel Bicycle Model	23
3.3 Nonlinear Bicycle Model	25
3.4 Steady State Dynamics	26
3.5 Transient Dynamics	30
4 Conclusion and Appendices	31
4.1 Conclusion	31
4.2 References	32

1.1 Description

This Study is in association with the Mississippi State Society of Automotive Engineers (SAE) Formula Car. The car is broken into different areas or disciplines called sections. These sections include Aerodynamics, Business, Chassis, Controls, Drivetrain, Electrical, Powertrain, and Suspension. This design independent study (DIS) focuses on the Suspension Section. Specifically, this study will focus on mathematically modeling the different aspects of suspension including: wishbone geometry, steering geometry, damping geometry and the dynamic equations that govern the handling.

1.2 Literature Review

There are many sources of information that are being drawn from when deriving the equations and designing the model and will be listed below. The main sources of information are: Derek Seward's *Race Car Design*, Milliken & Milliken's *Race Car Vehicle Dynamics*, Thomas D. Gillespie's *Fundamentals of Vehicle Dynamics*, and Pacejka's *Tire and Vehicle Dynamics*. These books are listed in order from introductory to advanced modeling.

Other source of information is I will attach my GitHub link that has python models of all that I will be talking about and referencing throughout this paper:

<https://github.com/ColinSpilker/FSAE-Suspension-Calc>.

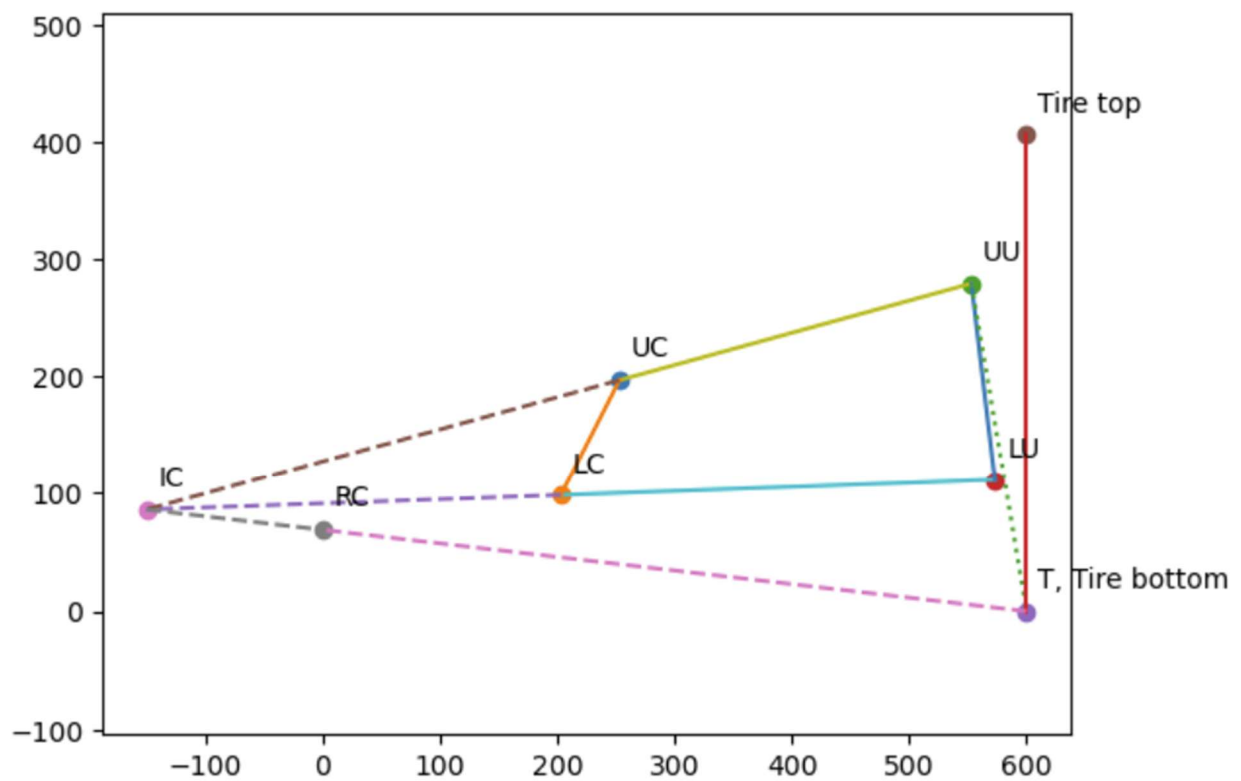
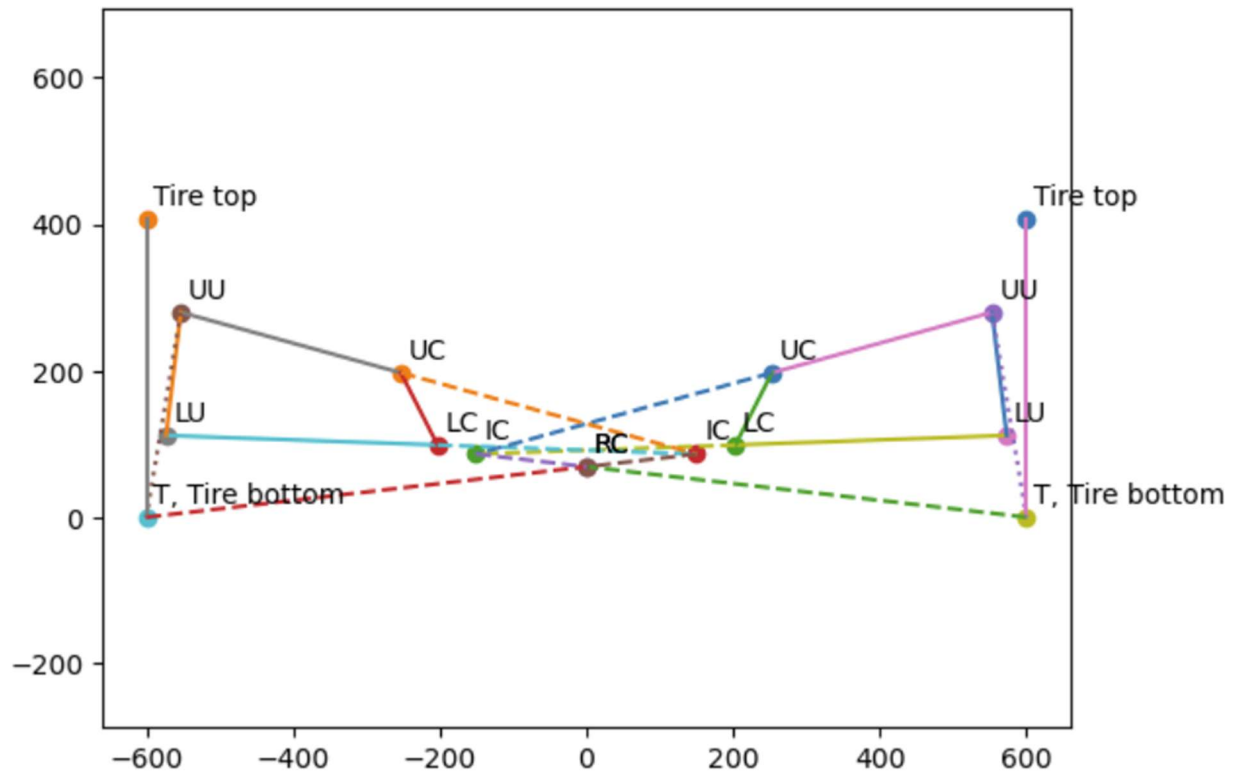
1.3 Purpose, Benefits, and Objectives

The main purpose of this is to properly model a nonlinear suspension geometry and the nonlinear dynamics and to understand how this subsystem works and predict how it may behave. The benefits will outlast my tenure on the team and provide a basis of foundation for understanding vehicle dynamics that are central to this car. The objectives I have set out include displaying and interpreting how these subsystems are modeled, what assumptions are used, what

methods are used, and have a working model to base off of. As of November 2025, this has been successful for 2d control arm geometry, steering geometry, damping geometry, and steady state dynamics; I have not yet completed transient dynamics but have an outline of what needs to be done and will include that.

2.1 Kinetic modeling

To determine the different ratios based on geometry, the geometry needs to be modeled. A front view 2D assumption will be implemented to easily solve with the geometry and this is valid as there is not much 3D anti-geometry. So this system takes the point values and plots to this:



Python simulated front view Suspension

Each point is labeled with chassis points, upright points, and tire points. The geometry intersection of the imaginary extension of the control arm lines is what is called the instant center which tells the instantaneous radius of the tires. The continuation of this idea with the intersection of the lines from instant center to the bottom of the tire is called the roll center. The roll center is used for both calculating sprung weight transfer and quantifying how much transient nonlinearities there are. This setup for geometry allows for iteration by moving the wheel up in the Z direction or rotating the chassis in the ϕ direction and solving for the angles.

Taking the suspension geometry to be a 4 bar linkage or a set of 2 connected 4 bar linkages, all the angles can be calculated and different motion ratios can be found. This is done by vector addition (simplified): $\frac{Y}{Z} * Chassis\ length + \frac{Y}{Z} * Upper\ control\ arm\ length + \frac{Y}{Z} * Upright\ length - \frac{Y}{Z} * Lower\ control\ arm\ length + \frac{0}{Z} * wheel\ motion = 0$. Solving for the different angles can give the total amount of camber that is changed over heave. Here is a snapshot of the working code doing this:

```
# Solve 3-vector loop: LC + L + U + Upr + TUC = known chassis coordinates
def positionloop(guessinput, MM_i, knowns):
    (LLCo, UUCo, UprUUo) = guessinput
    (U, L, C, Upr, T, CLCo, To, TUU, TUC, TUCo) = knowns

    try:
        loop = np.zeros(3)
        loop[0] = C*cos(CLCo) + U*cos(UUCo) + Upr*cos(UprUUo) - L*cos(LLCo)
        loop[1] = C*sin(CLCo) + U*sin(UUCo) + Upr*sin(UprUUo) - L*sin(LLCo)
        loop[2] = U*sin(UUCo) + TUU*sin(UprUUo + To) - MM_i - TUC*sin(TUCo)
    except Exception as e:
        print("Math error in positionloop:", e)
        loop = [np.nan, np.nan, np.nan]
    return loop
```

Python snapshot of working code for heave motion

The same can be done for the roll motion by rotating the chassis angles and keeping the wheels constant height. This is more complicated as there needs to be a loop looping both sides of the suspension together and then vary the angles by degree “a”:

```
# Vector Loop constraints (position closure equations)
loop = np.zeros(11)
loop[0] = UCa[1] + Ctop*sin(a) - UCb[1]
loop[1] = -UCa[0] + Ctop*cos(a) - UCb[0]
loop[2] = LCa[1] + Cbot*sin(a) - LCb[1]
loop[3] = -LCa[0] + Cbot*cos(a) - LCb[0]
loop[4] = LCa[0] + C*cos(CLCo - a) - UCa[0]
loop[5] = LCa[1] + C*sin(CLCo - a) - UCa[1]
loop[6] = LCb[0] + C*cos(CLCo + a) - UCb[0]
loop[7] = LCb[1] + C*sin(CLCo + a) - UCb[1]
loop[8] = C*sin(CLCo + a) + Ctop*sin(a) - Cbot*sin(a) - C*sin(pi - CLCo + a)
loop[9] = C*cos(CLCo + a) + Ctop*cos(a) - Cbot*cos(a) - C*cos(pi - CLCo + a)
```

Python snapshot of working code for roll.

Since the kinematic calculations are solved without forces, The main external constraint that is used is to calculate is to keep the roll center height constant. This assumption can work since the roll center migration is much larger in the horizontal direction than in vertical direction. This is done in place of solving for spring rate and mass of the car and rotational inertia; A moment on the car will not change height that much compared to roll.

To solve for camber of roll it is needed to subtract or add which direction the roll is to get the resulting amount of camber. This net camber is what the camber in roll will be. This is illustrated in Giuseppe’s *Fundamentals of Vehicle Dynamics* by a diagram showing the net camber angle.

The total camber angle during cornering will be:

$$\gamma_g = \gamma_b + \phi \quad (6-50)$$

where:

γ_g = Camber angle with respect to the ground

γ_b = Camber angle of the wheel with respect to the body

ϕ = Roll angle of the vehicle

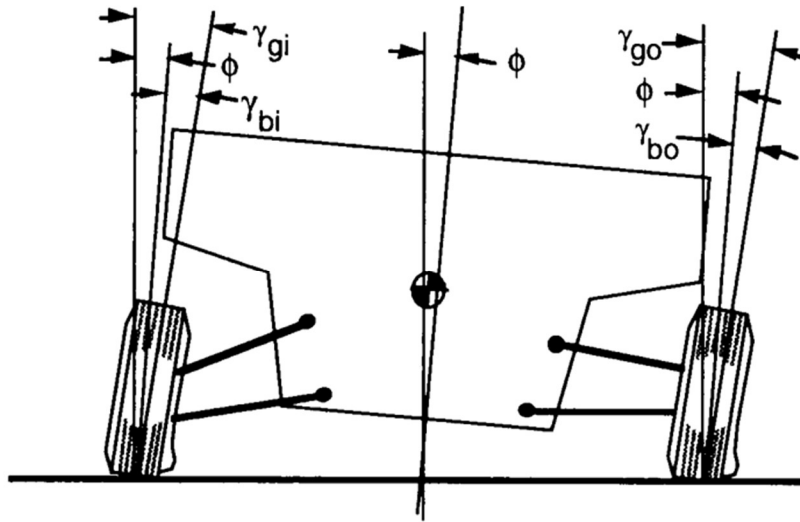
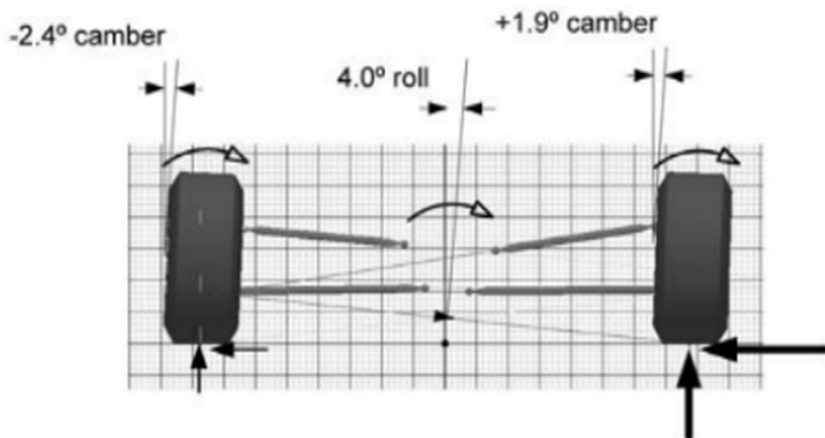


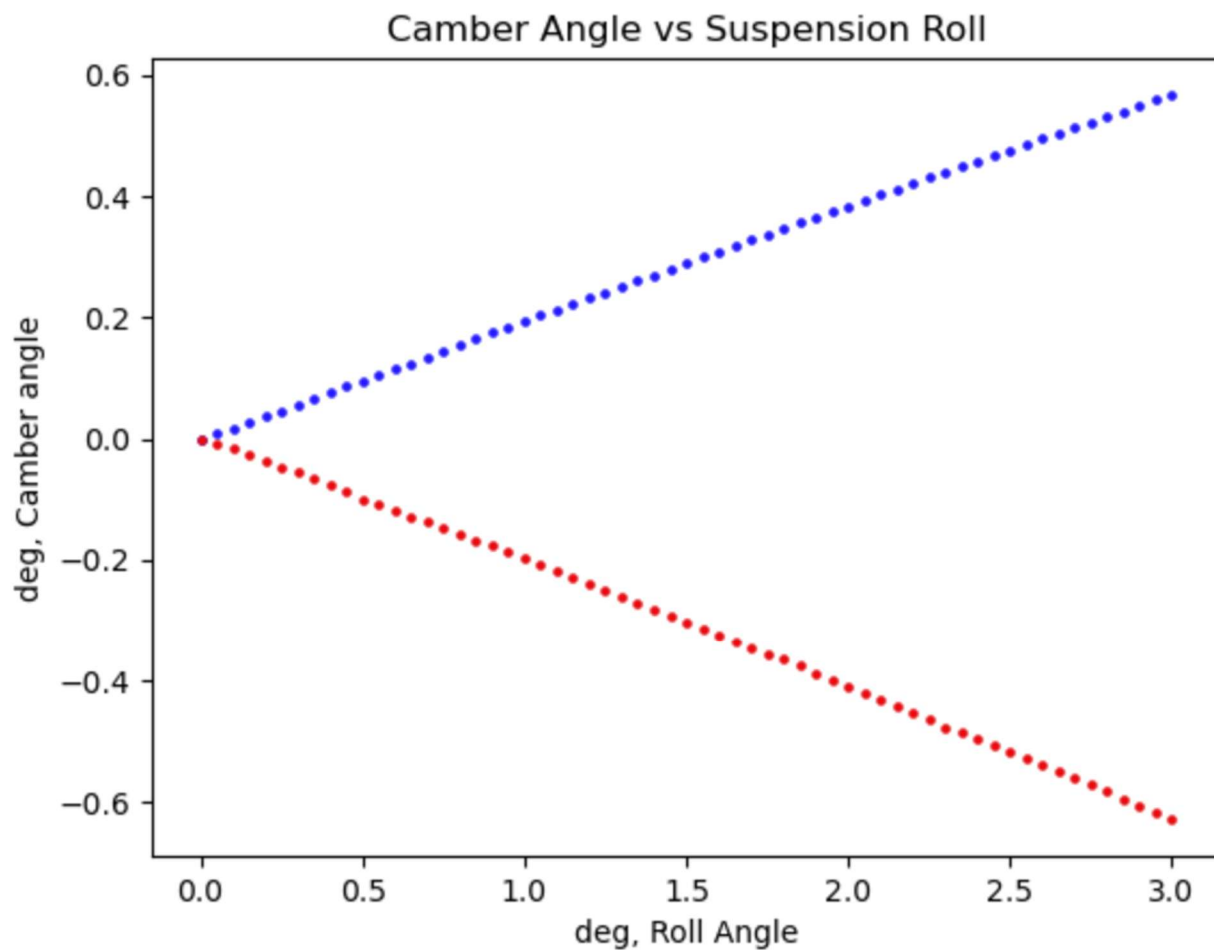
Fig. 6.15 Camber change in cornering of a vehicle.

(Milliken) illustration of camber in roll



(Seward) illustration of camber in roll

Camber is important in tuning the car, specifically in transient. As it is the first thing to be loaded and unloaded when changing cornering and movement as an ideal car is slightly overdamped. On this year's car, the targeted camber was high for heave but wanting to remain nearly constant throughout roll so there would not be large amounts of loading and unloading during cornering. This is the camber change over 3 degrees of roll with blue being outside and red being inside tire of the front tires as an example:



Python generated camber curves in roll

More negative camber tends to help during cornering as it applies a force in the same direction as the car is turning. Also keeping contact patch as larger as possible helps keep grip during slip. These two truths make tire behavior nonlinear, which is why having a near constant

roll-camber makes this a tuning problem not a design or kinematic problem; tuning will adjust to what the driver is most comfortable and what is tested to be the fastest as it is difficult to fully model tires.

2.2 Springs and Wheel Rates

To model the spring rates and wheel rates, A similar approach is used, except the bell crank damper subsystem needs to utilize 3D vectors instead of 2D vectors as this cannot be accurately approximated to 2D. Taking the pickup location from the control arms modeled in previous section, the relation of movement of pickup point can be added by adding Y and Z

values. The model will look like:
$$\frac{0}{Z} * Pickup\ points + \frac{X}{Z} * Push/Pullrod\ length + \frac{X}{Z} *$$

$$Bellcrank\ top\ length - \frac{X}{Z} * Damper\ length = 0$$
 with constraints on the bell crank to

prevent rotation. Here is the module for it in python:

```

def loop(vars,dY,dZ):
    X1,X2,X3,X4,X5,X6,X7,X8,X9, X10,X = vars

    loop=np.empty(11)

    loop[0]=(UprAtt[0] + d(X1,X2,0)*rod+ d(X3,X4,0)*bell3+ d(X5,X6,0)*(damp+X) - ChasAtt[0])
    loop[1]=(UprAtt[1]+ dY + d(X1,X2,1)*rod+ d(X3,X4,1)*bell3+ d(X5,X6,1)*(damp+X) - ChasAtt[1])
    loop[2]=(UprAtt[2]+ dZ + d(X1,X2,2)*rod+ d(X3,X4,2)*bell3+ d(X5,X6,2)*(damp+X) - ChasAtt[2])

    #Loop[3]=(d(X7,X8,0)*bell1+ d(X3,X4,0)*bell3- d(X9,X10,0)*bell2)
    #Loop[4]=(d(X7,X8,1)*bell1+ d(X3,X4,1)*bell3- d(X9,X10,1)*bell2)
    #Loop[5]=(d(X7,X8,2)*bell1+ d(X3,X4,2)*bell3- d(X9,X10,2)*bell2)

    loop[3]=(UprAtt[0] + d(X1,X2,0)*rod- d(X7,X8,0)*bell1 - BellPiv[0])
    loop[4]=(UprAtt[1]+ dY + d(X1,X2,1)*rod- d(X7,X8,1)*bell1 - BellPiv[1])
    loop[5]=(UprAtt[2]+ dZ + d(X1,X2,2)*rod- d(X7,X8,2)*bell1 - BellPiv[2])

    loop[6] =(BellPiv[0] +d(X9,X10,0)*bell2+ d(X5,X6,0)*(damp+X) - ChasAtt[0])
    loop[7]=(BellPiv[1] +d(X9,X10,1)*bell2+ d(X5,X6,1)*(damp+X) - ChasAtt[1])
    loop[8]=(BellPiv[2] +d(X9,X10,2)*bell2+ d(X5,X6,2)*(damp+X) - ChasAtt[2])

    bell1ar= np.array([d(X7,X8,0) *bell1,d(X7,X8,1) *bell1,d(X7,X8,2) *bell1])
    bell2ar= np.array([d(X9,X10,0)*bell2,d(X9,X10,1)*bell2,d(X9,X10,2)*bell2])
    b1pos = BellPiv + bell1ar
    b2pos = BellPiv + bell2ar
    b1norm=bell1ar/np.linalg.norm(bell1ar)
    b2norm=bell2ar/np.linalg.norm(bell2ar)
    bellCurr= np.cross(b1norm,b2norm)
    bellCurrNorm=bellCurr/np.linalg.norm(bellCurr)
    #Loop[9]= np.dot(bellCurrNorm, BellAxNorm)-1
    #Loop[10] = np.dot(b1pos - b2pos, b1pos - b2pos) - bell3**2
    #Loop[10]= 1000*(np.dot(b1norm, b2norm)- np.dot(B1norm, B2norm)) $better mathematically but r

    loop[9]= np.dot(bell1ar/ np.linalg.norm(bell1ar), BellAx / np.linalg.norm(BellAx))
    loop[10]= np.dot(bell2ar/ np.linalg.norm(bell2ar), BellAx / np.linalg.norm(BellAx)) #Does not
    #print(Z)
    return loop

```

Python coded damping loop equations to solve for motion ratio

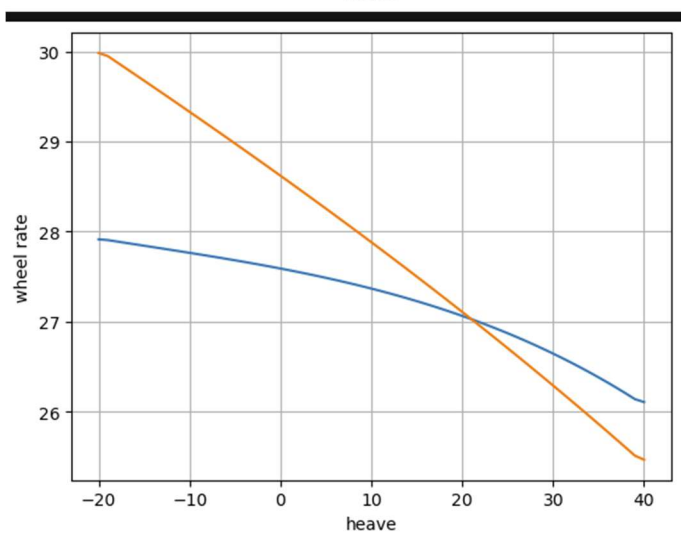
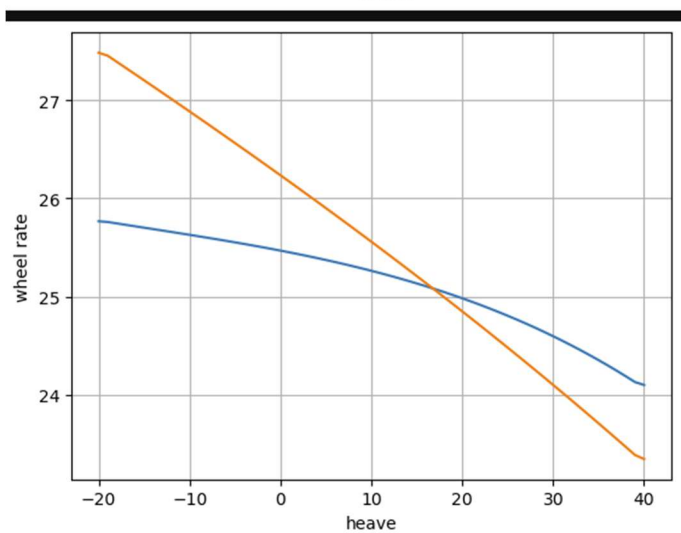
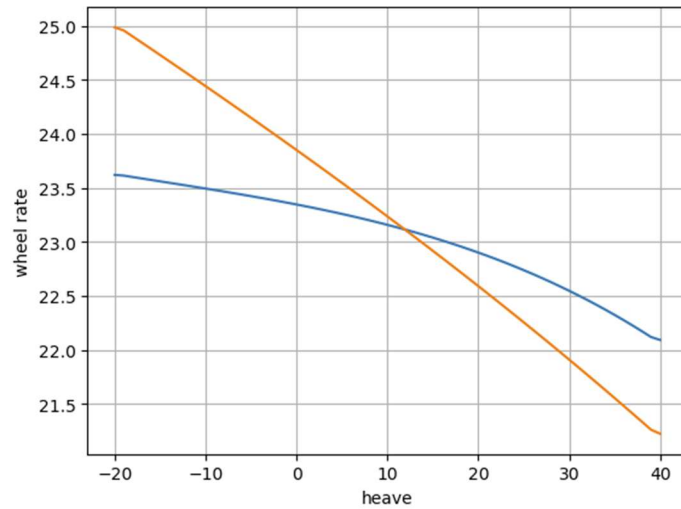
This will solve for the change in length of X which will denote the change of length of the spring damper. This will generate the motion ratio that is used when calculating wheel rates.

The motion ratio is calculated based on compression of the spring divided by heave of the wheel:

$IR = \frac{\delta_{Spring}}{Heave}$. Then the motion ratio is used to calculate the wheel rates by the eq: $K_w = K_s * (IR)^2$ [Milliken 596]

The wheel rates serve as the effective spring rates on each corner of the car

and act as total stiffness comparisons for the front and the rears. This comparison can determine bias on oversteer and understeer for the handling of the car. A stiffer front leads to less sprung weight transfer and more total weight transfer resulting in less grip and more understeer; same on the rears. This comparison is shown on this FSAE car by two graphs over heave to determine where the stiffness changes with front being orange, rear being blue:



Python plots showing wheel rates of front and rears with a 500/550 lb/in springs, 550/600 lb/in springs, and 600/650 lb/in springs in the front/rear.

2.3 Ackermann and Steering Geometry

Ackermann is the rate of change between both of the front steering wheels referring to when the inside tire is turning sharper than the outside tire with negative ackermann referring to the opposite. An 100% ackermann setup is when the tires do not slip during a radius of R length. R is variable for the purpose of the car and in our FSAE car, R is skidpad radius of 7.625m. There is an approximation of ackermann setup called a Jeantaud diagram that give an approximation of 100% Ackermann setup. Ackermann is implemented with using a linkage design to allow each wheel to move at different rates. This is done because the steering tie rod pickup point will be further or closer than the axis of rotation, causing greater or less than rotation. Moving the rack forward and backwards also changes this relation. A parallel setup will be neutral Ackermann or parallel steer. *Race Car Vehicle Dynamics* illustrates all of this well:

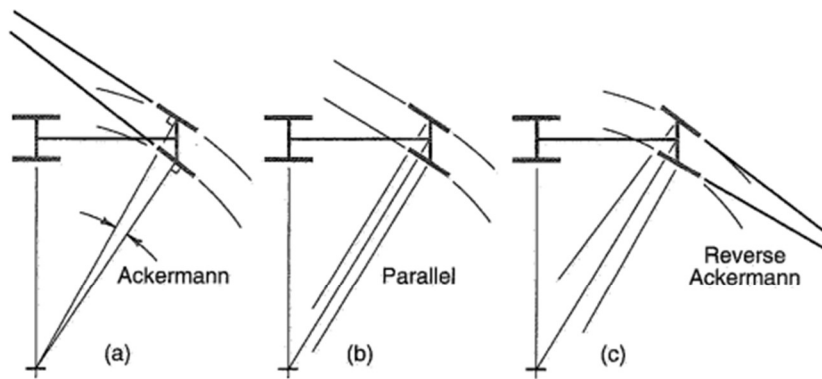


Figure 19.2 Ackermann steering geometry.

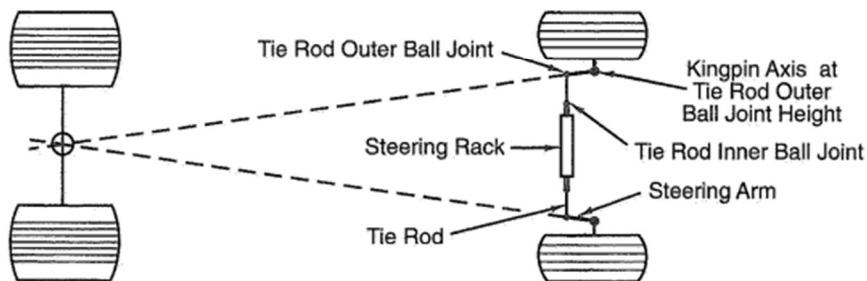


Figure 19.3 Ackermann geometry, with steering rack behind the axle line.

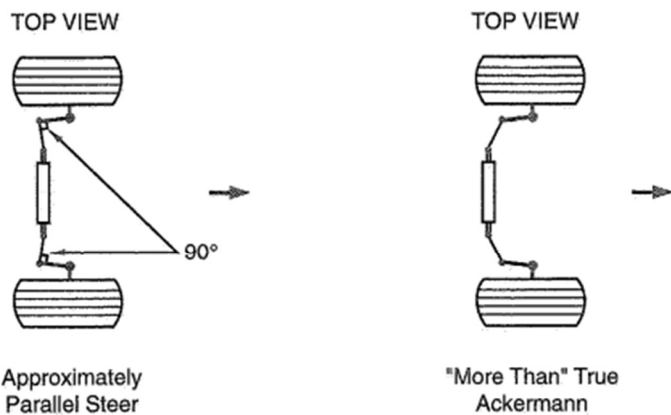
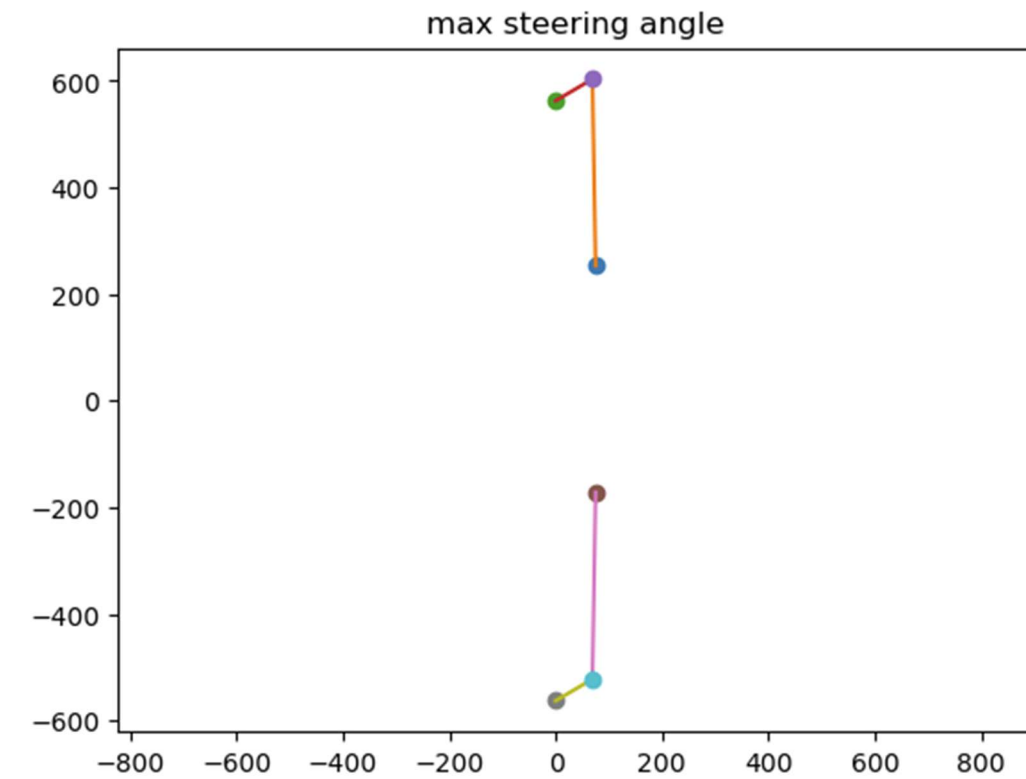
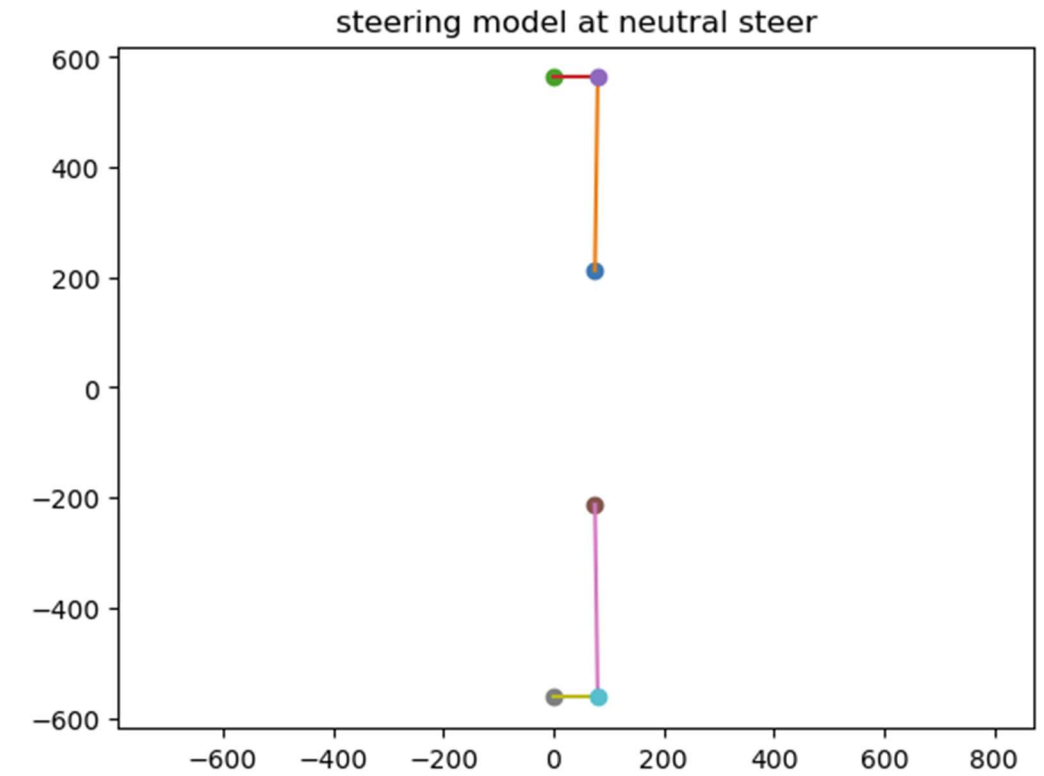


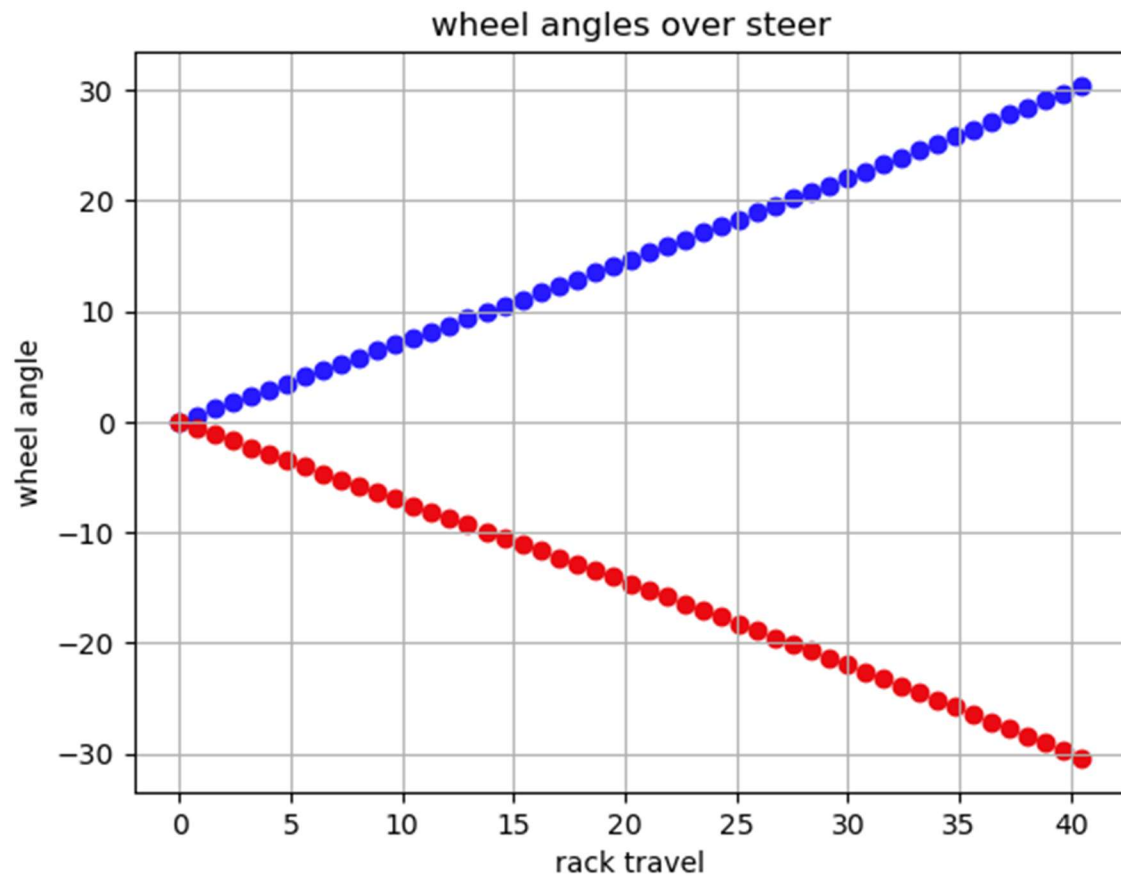
Figure 19.4 Modified steer geometry—moving steering rack forward and backward.

(Milliken) illustration depicting what Ackermann affects handling, a Jeantaud diagram and a tuning diagram.

Ackerman mostly affects grip via slip angle that will be talked more about in section 3.1. More ackermann percentage tends to drive and steer lighter in driver effort. More anti Ackermann has more slip and more grip, but it is not linear where there is a point of less return. In practicality, for an FSAE car, neutral ackermann or parallel steer or near these percentages perform the best. The other geometries for steering, like caster, kingpin, and pneumatic trail, has the most effect on driver feel and ergonomics rather than strict dynamics, so they were not modeled in this simulation. To model an accurate steering setup, the rack and movement needed to be placed and have the joints modeled as added vectors like in control arms and damping. This results in:



Python depiction of the steering geometry



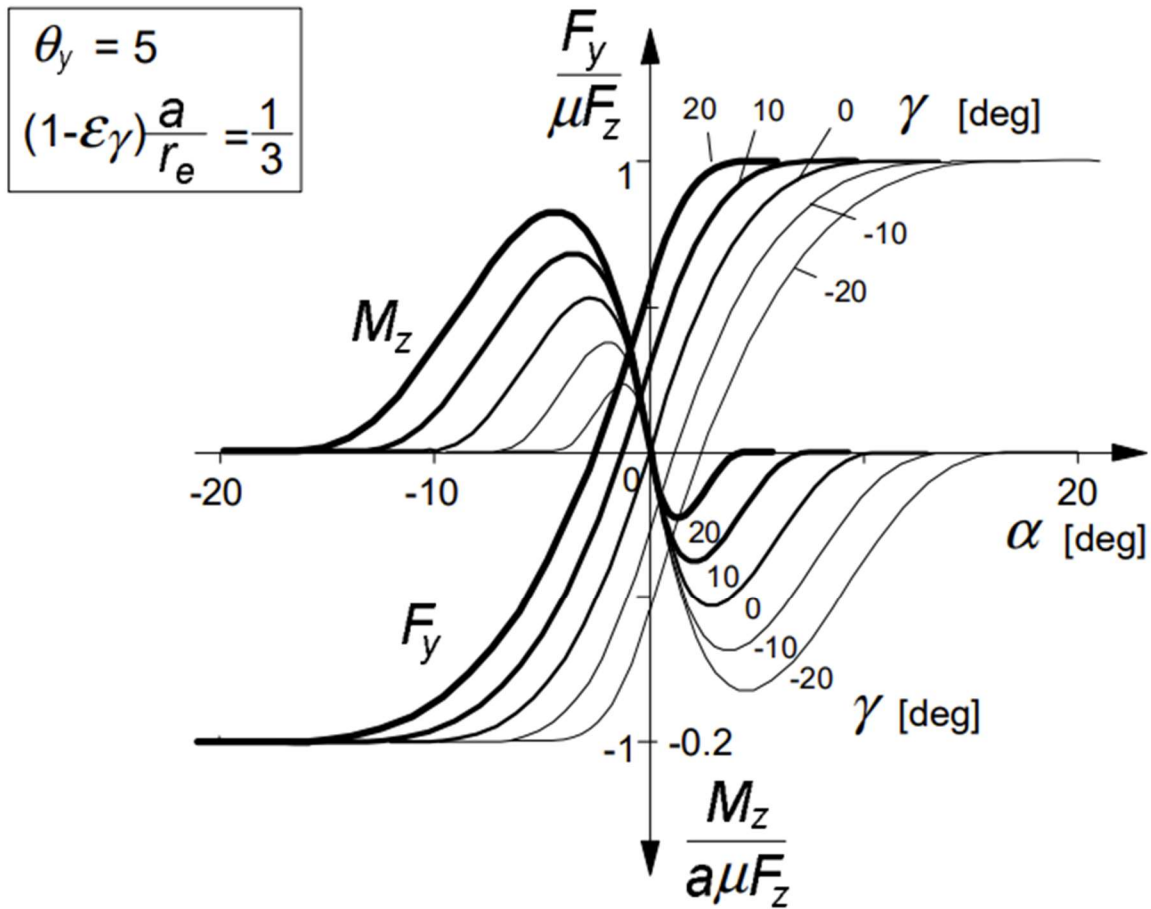
Python numerical study depicting the angles over travel giving angles at each mm of travel

3.1 Nonlinear tires

Tires are very nonlinear in behavior.

There are four main focal points of tires to look at for this FSAE analysis: slip angle, camber angle, and contact patch. Tires are non-Newtonian with loading and static friction. Tires do not follow the relationship of $F_s = \mu F_N$ but instead follow a trend of $F_s = \mu F_N^x$. The loading is not linear, and in most cases under isolation, $x < 1$. With more loading there is also an increase in contact patch because pneumatic tires deform shape under load which would increase x , so x would have to be solved empirically from data unless using the brush model which is a research material science rubber deformation model. Camber angle and contact patch are related as when increasing the magnitude of camber the tire is not gripping the ground as much, this is why having a flat roll camber leads to more numerical predictable grip.

Tire slip is the most researched and documented as there are different formulations of linearity to solve it. The linear model is inaccurate and depicts a $F_s = \mu \alpha$ with α being the slip angle. But after a certain amount of slip the increase in grip drops off, much like a stress strain curve of materials. This is shown by many different resources with this showing visually the relationship with slip angle and moment and force:



(Pacejka) displaying a graph of variable nonlinear forces and moments with respect to slip angle α , and camber γ .

Maximizing slip angle will give the best results to a point. Pacejka developed a formula called the Magic Tire Formula or Pacejka Formula which is able to map tire data to constants to create a smooth curve. The Magic Formula is:

Lateral Force (pure side slip)

$$F_{yo} = D_y \sin[C_y \arctan\{B_y \alpha_y - E_y(B_y \alpha_y - \arctan(B_y \alpha_y))\}] + S_{Vy} \quad (4.E19)$$

Aligning Torque (pure side slip)

$$M_{zo} = M'_{zo} + M_{zro} \quad (4.E31)$$

$$M'_{zo} = -t_o \cdot F_{yo} \quad (4.E32)$$

$$t_o = t(\alpha_t) = D_t \cos[C_t \arctan\{B_t \alpha_t - E_t(B_t \alpha_t - \arctan(B_t \alpha_t))\}] \cdot \cos' \alpha \quad (4.E33)$$

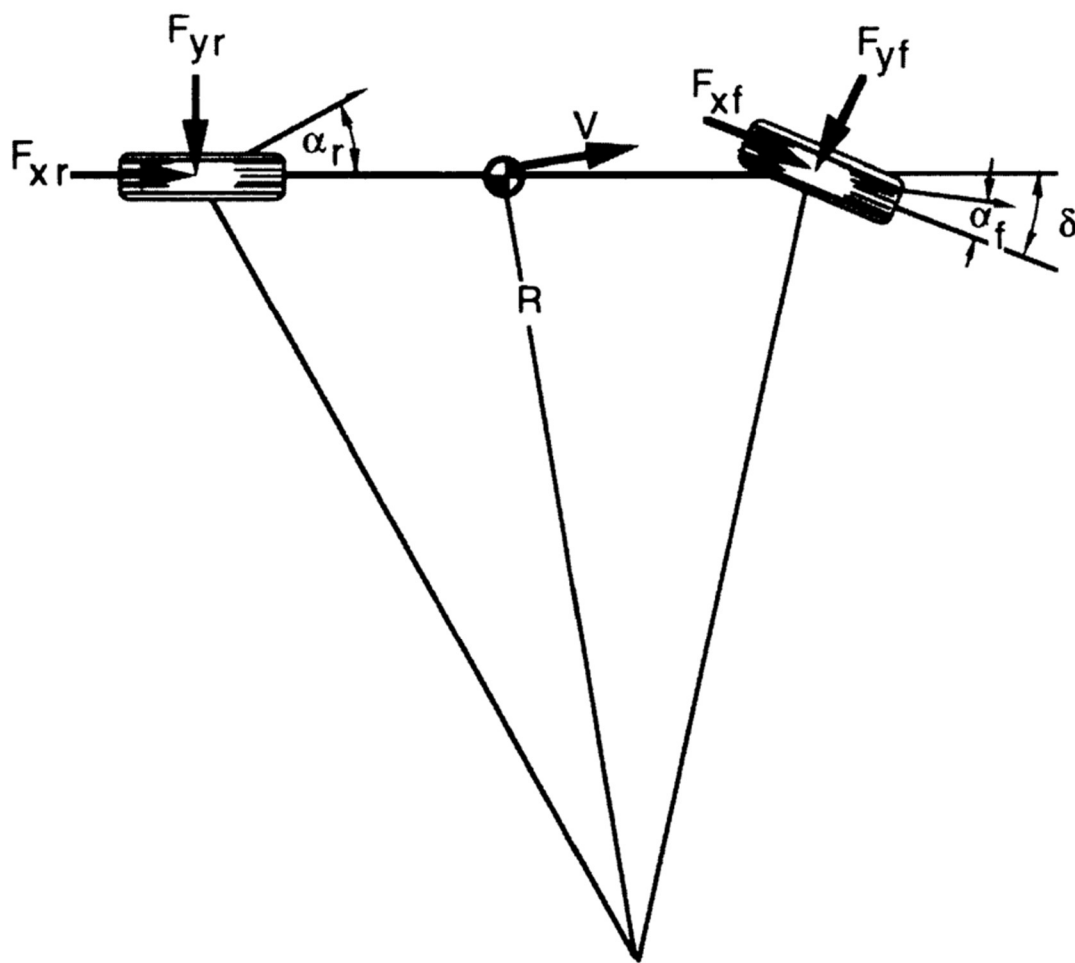
(Pacejka) Magic Formula for lateral force and moment

On these formulas, α is modified slip angle, B is stiffness factor, C is shape factor, D is peak value, E is curvature factors, S relates to camber and modifies α . D is related to μF_N^x making the loading and contact patch nonlinear. This is the full nonlinear empirical fit equations for most tires.

For the FSAE car we use a best fit line of tire data to determine proper force from slip angle to use in dynamics model. We generated our own empirical magic formula fits from TTC data and are using that in our usable results.

3.2 Linear 2 Wheel Bicycle model

The linear two-wheel dynamics model is a model to determine lateral force at the CG based off slip angle. There is a vector of speed at angle beta from direction of moving forward, this adjusted velocity vector applies an angle to both front and rear wheels, which generate force perpendicular to wheels direction of travel, the diagram below gets this incorrect with the radius lines but right with the forces. The diagram shows this conceptually below:



(Giuseppe) showing a conceptual diagram to show how a two-wheel model works.

With this being a linear model, this remains in the linear region of tire slip represented as $F_y = c\alpha$ and the c value is the stiffness coefficient for the front and rear based on contact patch,

camber, tire grip, weight loading and other variables that affect the dynamics. Using stiffness, this can generate a understeer oversteer gradient, K. This is shown by the different variables below:

UNDERSTEER COMPONENT

$$K_{\text{tires}} = \frac{W_f}{C_{\alpha f}} - \frac{W_r}{C_{\alpha r}}$$

$$K_{\text{camber}} = \left(\frac{C_{\gamma f}}{C_{\alpha f}} \frac{\partial \gamma_f}{\partial \phi} - \frac{C_{\gamma r}}{C_{\alpha r}} \frac{\partial \gamma_r}{\partial \phi} \right) \frac{\partial \phi}{\partial a_y}$$

$$K_{\text{roll steer}} = (\epsilon_f - \epsilon_r) d\phi/da_y$$

$$K_{\text{lfcS}} = A_f W_f - A_r W_r$$

$$K_{\text{at}} = W \frac{p}{L} \frac{C_{\alpha f} + C_{\alpha r}}{C_{\alpha f} C_{\alpha r}}$$

$$K_{\text{llt}} = \frac{W_f}{C_{\alpha f}} \frac{2 b \Delta F_{zf}^2}{C_{\alpha f}} - \frac{W_r}{C_{\alpha r}} \frac{2 b \Delta F_{zr}^2}{C_{\alpha r}}$$

$$K_{\text{strg}} = W_f \frac{r v + p}{K_{ss}}$$

SOURCE

Tire cornering stiffness

Camber thrust

Roll steer

Lateral force compliance steer

Aligning torque

Lateral load transfer

Steering system

(Giuseppe) showing the understeer effects and understeer gradient.

These understeer effects are based on linear approximation and are useful but do not scale to the full nonlinear system and need a transient model to properly measure the understeer effects more accurately.

3.3 Nonlinear Bicycle Model

Compared to the linear bicycle model the 4 wheel model is nonlinear and heavily coupled. Having four wheels implies having to solve for each wheel slip by applying beta to each new frame of reference and solving for weight transfer. This means that to solve, all of this needs to be coupled and use a nonlinear Newtonian method to solve rather than solving all analytically.

The transform applied to each wheel is:
$$\begin{bmatrix} \cos(\pm\psi_{steer} \pm \psi_{toe}) & \sin(\pm\psi_{steer} \pm \psi_{toe}) \\ -\sin(\pm\psi_{steer} \pm \psi_{toe}) & \cos(\pm\psi_{steer} \pm \psi_{toe}) \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix}.$$

Sum of moments and forces have to take values from all wheels making coupling complicated quickly. It changes a 2DOF or 3DOF system to a 5,6 DOF system by adding Y movement, and ϕ roll and possibly θ pitch. The difference between 2 and 3DOF and 4 and 5DOF is the inclusion of aero normal forces, which my model has corresponded with the aero sub-team and gotten center of pressure and the downforce calculations approximations. Here are the equations, simplified as a 5DOF: and a diagram:

$$F_i = D * \sin (C \arctan(B * (\alpha + S_h) - E(B(\alpha + S_h) - \arctan(\alpha + S_h)))) + S_v$$

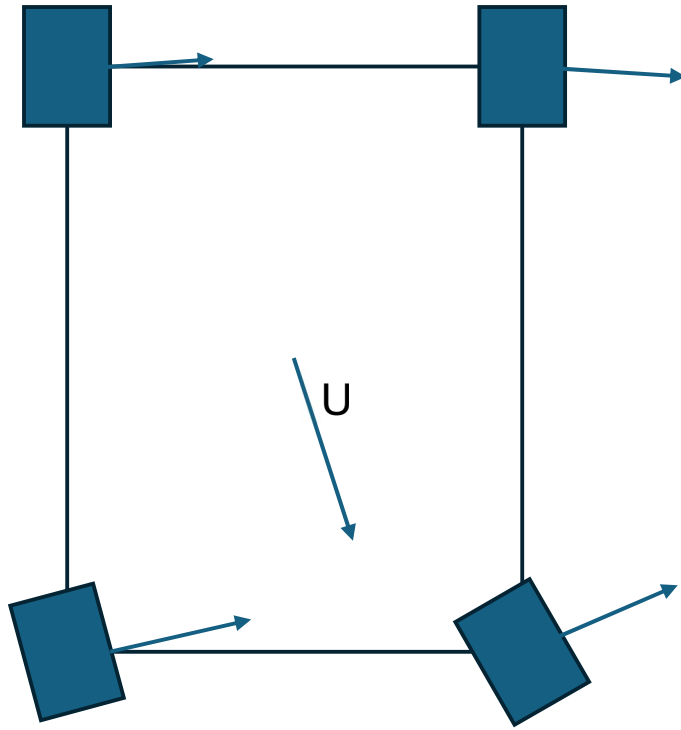
$$M_{z_{wheel}} = -F_i * t + M_i : \text{where } t \text{ and } M_i = F_{i_{eq}} \text{ with different parameters}$$

$$D \sim (\phi K_\phi / K_{norm,z} * (F_{aero} + mg * (CG_x/l, \quad 1 - CG_x/l)) / 4)^n$$

$$\alpha$$

$$= \arctan \left(\frac{-\sin(\pm\psi_{steer} \pm \psi_{toe}) (\dot{X} \pm \dot{\psi} (CG_x, l - CG_x)/2) + \cos(\pm\psi_{steer} \pm \psi_{toe}) (\dot{Y} \pm \dot{\psi} w/2)}{\cos(\pm\psi_{steer} \pm \psi_{toe}) (\dot{X} \pm \dot{\psi} (CG_x, l - CG_x)/2) + \sin(\pm\psi_{steer} \pm \psi_{toe}) (\dot{Y} \pm \dot{\psi} w/2)} \right)$$

$$S_h, S_u \sim \pm \phi \text{ and } \pm Z \text{ from camber}$$



Equations of 4 wheel transformation with a diagram of model

3.4 Steady State Dynamics

For steady state dynamics, this implies constant speed cornering, where all forces are equal to 0 aside from lateral acceleration as it is driving in a circle. In my 5DOF model, these are the listed equations for acceleration and velocity of all values:

$$\ddot{X} = - \sum F_{i_{wheel}} \sin(\psi_{wheel}) / m + F_x / m = 0$$

$$\dot{X} = \sin(\psi) * U$$

$$X = 0$$

$$\ddot{Y} = \sum F_{i_{wheel}} \cos(\psi_{wheel}) / m - U * \dot{\psi} / m = 0$$

$$\dot{Y} = \cos(\psi) * U$$

$$Y = 0$$

$$\ddot{Z} = (F_{aero}[U] + mg - \sum F_{N_{wheel}})/m = 0$$

$$\dot{Z} = Z = 0$$

$$\ddot{\Psi} = \left(\sum \pm F_{i_{wheel_x}} * w/2 \pm F_{i_{wheel_y}} * l * (CG_x/l) + \sum M_{z_{wheel}} \right) / I_z = 0$$

$$\dot{\Psi} = r = U/R$$

$$\Psi = \beta$$

$$\ddot{\Phi} = \dot{\Phi} = 0$$

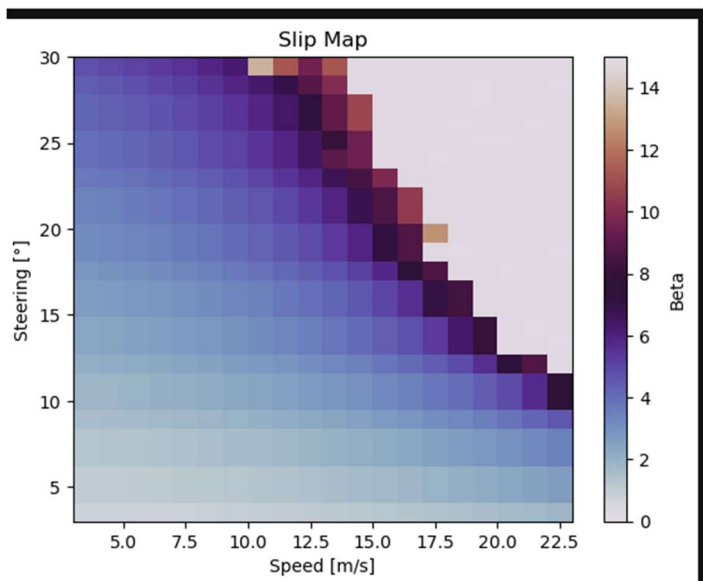
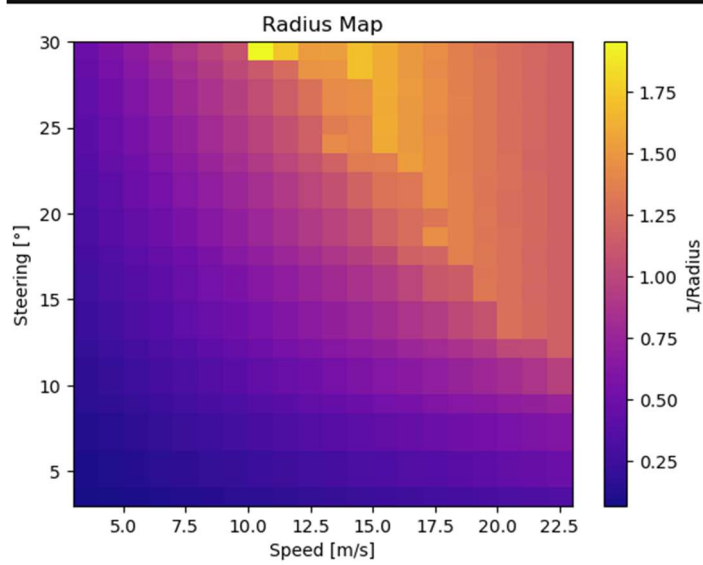
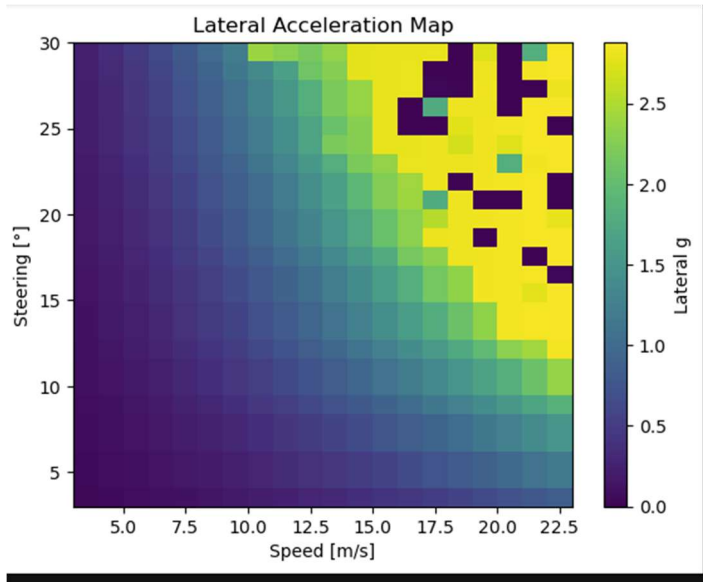
$$\Phi = m * \ddot{Y} * (CG_z - RC_z)/K_{roll}$$

$$\ddot{\Theta} = \dot{\Theta} = \Theta = 0$$

$$\textit{Given: } m, U, F_{aero}[U], CG_x, \Psi_{steer}$$

$$\textit{Free: } \Phi, (r \text{ or } 1/R), (\beta = \Psi)$$

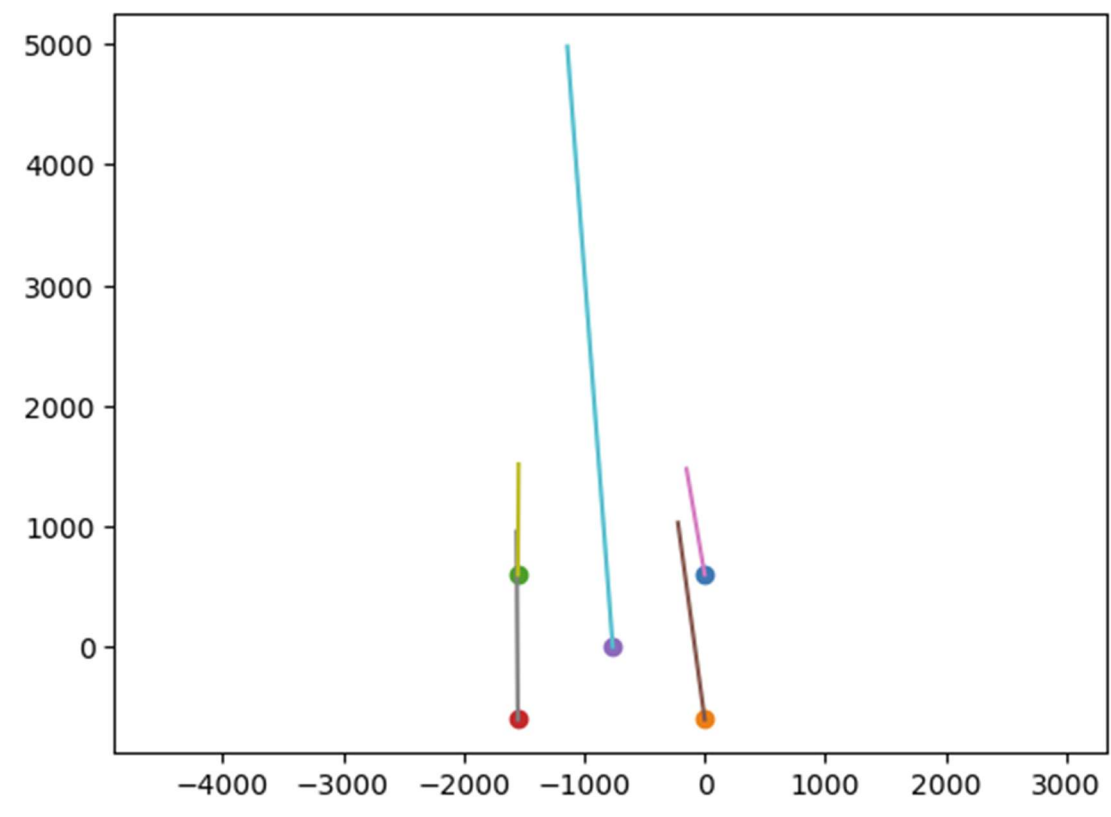
The solver is set up to output angle, force, radius, and roll. Using these values, the speed map to tell what a setup on a car will perform can be made. Here is a map of range of values from this car's setup that will be used to predict behavior:



Python plotting all values for variable speed and steering percentages resulting in a map telling what force and corner radius is possible, and when the car will slip and either go to over or understeer.

This model is inaccurate at very low speeds where there is very low slip because it does not follow geometric curve of zero slip. This means it is accurate at high speeds and high slip but not a rolling car around a parking lot.

There is also plotting of a single speed and steering to see if any tire is being overly loaded and can test to see if a single tire will fail. It can also output a free body diagram of the multiple bodies:



Python plot of free body diagram with magnitude of lines resulting in how strong the force is, car moving +X and turning +Y. Center is CG and sum of forces.

This serves as a tuning bed for stiffness of spngs and wheel loading and general geometry. Camber is more applicable in transient as it has the leading order for loading and unloading tires.

3.5 Transient Dynamics

My software package did not solve for transient yet, but what transient would be is listed below:

$$\ddot{\psi}I + \dot{\psi} \frac{\partial M_z}{\partial \dot{\psi}} + \psi \frac{\partial M_z}{\partial \psi} = \sum \pm F_{i_{wheel_x}} * w/2 \pm F_{i_{wheel_y}} * l * (CG_x/l) + \sum M_{z_{wheel}}$$

$$\ddot{\phi}I + \dot{\phi}C + \phi K = m\ddot{Y} * (CG_z - RC_z)$$

$$\ddot{X}_m = - \sum F_{i_{wheel}} \sin(\psi_{wheel}) + F_x$$

$$\dot{X} = u \cos(\psi) - v \sin(\psi)$$

$$\ddot{Y}_m = \sum F_{i_{wheel}} \cos(\psi_{wheel}) - U\dot{\psi}$$

$$\dot{Y} = u \sin(\psi) + v \cos(\psi)$$

$$\ddot{Z}_m + \dot{Z}C + ZK = F_{aero} + mg$$

$$\alpha$$

$$= \arctan \left(\frac{-\sin(\pm\psi_{steer} \pm \psi_{toe}) (u \pm \dot{\psi} (CG_x, l - CG_x)/2) + \cos(\pm\psi_{steer} \pm \psi_{toe}) (v \pm \dot{\psi} w/2)}{\cos(\pm\psi_{steer} \pm \psi_{toe}) (u \pm \dot{\psi} (CG_x, l - CG_x)/2) + \sin(\pm\psi_{steer} \pm \psi_{toe}) (v \pm \dot{\psi} w/2)} \right)$$

4.1 Conclusion

This software package serves as a basis for design for this year's car, serves as a tuning bed for us to make educated guesses about what aspects need to be adjusted for a given result, and as a basis of understanding for the design competition. This is planned to be expanded and used for years to come.

4.2 References

Gillespie, T. D., et al. *Fundamentals of Vehicle Dynamics*. SAE International, 2021.

Millikin, William., Millikin, Douglas. *Race Car Vehicle Dynamics*. Society of Automotive Engineers. 1995. Warrendale, PA.

Pacejka, Hans B. *Tire and Vehicle Dynamics*. SAE International, 2006.

Seward, Derek. *Race Car Design*. Palgrave. 2014. London, UK.

<https://github.com/ColinSpilker/FSAE-Suspension-Calc>.